FusionSystems

Fusion systems in GAP

0.1

21 October 2026

Pete Gautam
Justin Lynd

Pete Gautam Email: pratyush.gautam@manchester.ac.uk
Homepage: https://github.com/pete-g00
Address: Alan Turing Building,
University of Manchester,
Manchester,
M13 9PY,
United Kingdom

Justin Lynd Email: lynd@louisiana.edu
Homepage: https://userweb.ucs.louisiana.edu/~jxl7583/
Address: Mathematics Department
University of Louisiana at Lafayette
217 Maxim Doucet Hall, P.O. Box 43568
Lafayette, LA 70504-3568 USA

Abstract

The FusionSystems package allows for computation with fusion systems. Currently, it supports finding proto-essential subgroups for a given p-group and checking whether it could support reduced fusion systems.

Copyright

FusionSystems is free software; you can redistribute and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version.

Acknowledgements

We thank Martin van Beek for helping expand the list of proto-essential and reduced checks. We also thank David Burrell and Josh Bridges for their contributions to the project.

Contents

1 Introduction **4**

2 Proto-essential subgroups **5**
2.1 Brief description of the algorithm 5
2.2 The operations 5

3 Checks for proto-essentials **7**
3.1 The main functions 7
3.2 The specific functions 8

4 Checks for reduced fusion systems **13**
4.1 The main functions 13
4.2 The specific functions 14

5 Automizer Sequences **21**
5.1 The functions 21

6 Other functions **23**
6.1 Group actions 23
6.2 Group automorphisms 25
6.3 Semi-characteristic subgroups 26

References **29**

Index **30**

Chapter 1

Introduction

The FusionSystems package tests whether saturated fusion systems with given properties could exist on a finite p-group S, without constructing them. Its main functions are:

- `AllProtoEssentials` (2.2.1) finds the subgroups of S that could be essential in a saturated fusion system on S, up to S-conjugacy, and `IsProtoEssential` (3.1.1) tests a single subgroup (Chapters 2 and 3);
- `PossiblySupportsReducedFusionSystems` (4.1.1) tests whether S could support a reduced fusion system (Chapter 4);
- `FindReducedSequences` (5.1.1) finds the automizer sequences that could lead to reduced fusion systems (Chapter 5).

The tests check necessary conditions only: a subgroup found to be proto-essential, or a group that passes the tests for reduced fusion systems, need not lead to an actual saturated fusion system.

To follow the progress of a longer computation, set the info level of `InfoFusion` to 1 or higher.

Example

```
gap> S := SmallGroup(2^7, 931);;
gap> Length(AllProtoEssentials(S));
5
gap> PossiblySupportsReducedFusionSystems(S);
true
```

Chapter 2

Proto-essential subgroups

2.1 Brief description of the algorithm

Let S be a finite p-group. Recall that E is proto-essential in S if there *could* exist a saturated fusion system $\mathscr{F}$ on S such that $E \in \mathscr{E}(\mathscr{F})$. See Chapter 3 for the list of specific tests currently implemented. In this section, we describe the algorithm for constructing all the proto-essential subgroups of S. This is based on [Gau26, Section 3]. We fix a central series of S:

$$1 = Z_0 \leq Z_1 \leq \ldots \leq Z_n = S,$$

where $|Z_i| = p^i$ for $1 \leq i \leq n$, and there exists some $1 \leq j \leq n$ such that $Z_j = S'$. For $0 \leq i < j$, let $\pi_i : S \to S/Z_i$ be the projection map, and

$$\mathfrak{C}_i := \{E \mid E = \pi^{-1}(C_S(x)), x \in S/Z_i \text{ of order } p \text{ with } E \text{ proto-essential in } S\}.$$

We let $\mathfrak{C} := \bigcup_{i=0}^{j} \mathfrak{C}_i$. In [Gau26, Theorem 3.6], Gautam shows that if $\mathscr{F}$ is a saturated fusion system on S with $E \in \mathscr{E}(\mathscr{F})$, then E is $\mathscr{F}$-conjugate to some $X \in \mathfrak{C}$.

2.2 The operations

2.2.1 AllProtoEssentials (for IsPGroup)

▷ `AllProtoEssentials(`S`)` (attribute)

Returns: a list

The attribute `AllProtoEssentials` constructs all proto-essential subgroups of a p-group S. It works as follows:

- it calls `MainProtoEssentials` (2.2.2) which constructs the set $\mathfrak{C}$;
- it then calls `GenerateProtoEssentials` (2.2.3), which uses $\mathfrak{C}$ to construct all proto-essential subgroups, switching from $\mathscr{F}$-conjugacy to S-conjugacy (independent of $\mathscr{F}$).

It should be pointed out that there is no known cases where `GenerateProtoEssentials` generates a further (S-conjugacy) class of proto-essential subgroups. Finds all the proto-essential subgroups of S, up to S-conjugacy.

Example

```
gap> S := SmallGroup(3^4, 7);
<pc group of size 81 with 4 generators>
gap> L := AllProtoEssentials(S);
[ Group([ f4, f1, f3 ]), Group([ f4, f2, f3 ]), Group([ f1, f4 ]) ]
gap> List(L, StructureDescription);
[ "(C3 x C3) : C3", "C3 x C3 x C3", "C3 x C3" ]
```

2.2.2 MainProtoEssentials

▷ MainProtoEssentials(*S[, i]*) (function)

Returns: a list

Returns the list of the main proto-essential subgroups of S. A proto-essential subgroup is called *main* if it lies in $\mathfrak{C}$.

If the second argument i is provided, the function behaves as follows:

- $i = 0$ -- we only compute $\mathfrak{C}_0$;
- $i = -1$ -- we only compute $\mathfrak{C} \setminus \mathfrak{C}_0$; and
- $i = 1$ -- we compute all $\mathfrak{C}$.

If the argument is not provided, we compute all $\mathfrak{C}$. Running GenerateProtoEssentialson the list L returned generates all the proto-essentials subgroups of S.

Example

```
gap> S := SmallGroup(2^7, 931);
<pc group of size 128 with 7 generators>
gap> C0 := MainProtoEssentials(S, 0);
[ Group([ f1, f2, f4, f6, f7 ]), Group([ f1, f3, f5, f6, f7 ]),
Group([ f2, f3, f4, f5, f6, f7 ]) ]
gap> C := MainProtoEssentials(S, -1);
[ Group([ f7, f1, f2, f4, f5, f6 ]), Group([ f7, f1, f3, f4, f5, f6 ]) ]
```

2.2.3 GenerateProtoEssentials (for IsPGroup, IsList)

▷ GenerateProtoEssentials(*S, L*) (operation)

Returns: a list

Uses the list L containing the main proto-essential subgroups of S (i.e. the set $\mathfrak{C}$) to generate all the proto-essential subgroups of S.

Example

```
gap> S := SmallGroup(3^6, 321);
<pc group of size 729 with 6 generators>
gap> L := MainProtoEssentials(S);
[ Group([ f6, f2, f3, f4, f5 ]), Group([ f6, f1*f2^2, f4, f5 ]) ]
gap> L := GenerateProtoEssentials(S, L);
[ Group([ f6, f2, f3, f4, f5 ]), Group([ f6, f1*f2^2, f4, f5 ]) ]
```

Chapter 3

Checks for proto-essentials

In this chapter, we describe all the tests currently implemented in GAP that test whether a subgroup E of a p-group S is proto-essential. Recall that E is proto-essential in S if there *could* exist a saturated fusion system $\mathscr{F}$ on S such that $E \in \mathscr{E}(\mathscr{F})$.

3.1 The main functions

3.1.1 IsProtoEssential (for IsPGroup, IsPGroup)

▷ `IsProtoEssential(S, E)` (operation)

Returns: true or false

Let S be a p-group and E be a subgroup of S. The operation `IsProtoEssential` checks whether E is a proto-essential subgroup of S.

This function checks whether the group E satisfies certain properties with respect to S that are known to hold when E is an essential subgroup in some fusion system $\mathscr{F}$ on E. In this process, we construct (in order):

- $C_S(E)$;
- $N_S(E)$;
- $\mathrm{Aut}(N_S(E))$; and
- $\mathrm{Aut}(E)$.

Each computation is considered more expensive than the previous one. See Section 3.2 for the list of tests currently implemented.

Note that it is possible that E is found to be proto-essential even though there is no fusion system $\mathscr{F}$ on S with $E \in \mathscr{E}(\mathscr{F})$. This can be for various reasons, for example, E could be essential in $\mathscr{F}$ only if there is an overgroup of E that is also essential, but there is no such group. Some of these cases may be ruled out using automizer sequences, but it cannot be fully determined without constructing the actual fusion systems.

Example

```
gap> S := SmallGroup(3^4, 7);
<pc group of size 81 with 4 generators>
gap> C := SubgroupsSolvableGroup(S);;
gap> C := Filtered(C, A -> IsProtoEssential(S, A));
```

```
[ Group([ f3, f4, f1 ]), Group([ f1, f4 ]), Group([ f3, f4, f2 ]) ]
gap> List(C, StructureDescription);
[ "(C3 x C3) : C3", "C3 x C3", "C3 x C3 x C3" ]
```

3.1.2 IsProtoEssentialNC (for IsPGroup, IsPGroup)

▷ `IsProtoEssentialNC(S, E)` (operation)

Returns: true or false

The operation `IsProtoEssentialNC` behaves like `IsProtoEssential`, but further assumes that:

- E is a proper subgroup of S;
- $C_S(E) \leq E$;
- E passes `Zed0Test`.

3.2 The specific functions

3.2.1 Zed0 (for IsPGroup)

▷ `Zed0(S)` (attribute)

Returns: a subgroup

Let S be a p-group. The attribute `Zed0` computes the subgroup Z_0 of S, which is given by:

$$Z_0 := \langle x \in Z_2(S) \mid |[S,x]| \leq p \rangle.$$

3.2.2 Zed0Test (for IsPGroup, IsPGroup)

▷ `Zed0Test(S, E)` (operation)

Returns: true or false

Let S be a p-group and E be a subgroup of S. The operation `Zed0Test` checks that one of the following holds:

- $Z_0 \leq E$; or
- $[N_S(E) : E] = p$, and there exists some $x \in \Omega(Z(E))$ such that $E = C_S(x)$.

Note that this function is not used when constructing $\mathfrak{C}$ as the choice of x is easier to find while the set $\mathfrak{C}_0$ is being constructed.

This test applies at any prime p. It is based on [AOV17, Proposition 2.3(a)] (proven for $p = 2$).

Example

```
gap> S := SmallGroup(2^7, 931);
<pc group of size 128 with 7 generators>
gap> Z0 := Zed0(S);
Group([ f6, f7 ])
gap> C := SubgroupsSolvableGroup(S);;
gap> Length(C);
135
gap> Number(C, A -> Zed0Test(S,A));
66
```

3.2.3 RankTest (for IsPGroup, IsPGroup)

▷ `RankTest(S, E)` (operation)

Returns: a number

Let S be a p-group and E be a subgroup of S. The operation `RankTest` checks that $\mathrm{Out}_S(E)$ is isomorphic to a Sylow p-subgroup of a group containing a strongly p-embedded subgroup. If so, we return a value specifying its type, namely:

- 0 -- $\mathrm{Out}_S(E)$ is cyclic or quaternion;
- 1 -- $\mathrm{Out}_S(E)$ is elementary abelian of order $\geq p^2$;
- 2 -- Sylow p-subgroup of $\mathrm{PSU}_3(p^n)$;
- 3 -- Sylow 2-subgroup of $\mathrm{Sz}(2^{2n+1})$;
- 4 -- Sylow 3-subgroup of ${}^2\mathrm{G}_2(3^{2n+1})$; and
- 5 -- 3_-^{1+2} and 5_-^{1+2}.

We return -1 if none of the cases above apply. During the process, we also check that the subgroup E has sufficiently high rank to ensure that $\mathrm{Aut}(E)$ has a possible valid section.

This test applies at any prime p. It is based on [Sam14, Proposition 6.9].

Example

```
gap> S := SmallGroup(3^6, 479);
<pc group of size 729 with 6 generators>
gap> A := First(MaximalSubgroups(S), IsElementaryAbelian);
Group([ f2, f3, f4, f5, f6 ])
gap> B := First(MaximalSubgroups(S), A -> Exponent(A) = 3
>               and not IsElementaryAbelian(A));
Group([ f1, f3, f4, f5, f6 ])
gap> C := First(MaximalSubgroups(S), A -> Exponent(A) = 9);
Group([ f1, f2, f3, f5, f6 ])
gap> T := Intersection(A, B);
Group([ f3^2, f4^2, f5^2, f6^2 ])
gap> U := Intersection(A, B, C);
Group([ f3^2, f5^2, f6^2 ])
gap> RankTest(S, A);
0
gap> RankTest(S, B);
0
gap> RankTest(S, C);
0
gap> RankTest(S, T);
1
gap> RankTest(S, U);
-1
```

3.2.4 ModuleTest (for IsPGroup, IsPGroup)

▷ `ModuleTest(S, E)` (operation)

Returns: true or false

Let S be a 2-group and E be a subgroup of S. The operation `ModuleTest` checks that, if $\mathrm{Out}_S(E)$ has order 2^k (for $k \geq 2$), then for all $x \in \mathrm{Out}_S(E)$, if $x \neq 1$, then $[x, E/\Phi(E)]$ is not cyclic.

This test applies at $p = 2$. It is based on [OV09, Proposition 3.3(c)].

Example

```
gap> S := SmallGroup(2^7, 931);
<pc group of size 128 with 7 generators>
gap> C := SubgroupsSolvableGroup(S);;
gap> Length(C);
135
gap> Number(C, A -> Zed0Test(S,A) and ModuleTest(S,A));
24
```

3.2.5 FrattiniTest (for IsPGroup, IsPGroup)

▷ `FrattiniTest(S, E)` (operation)

Returns: true or false

Let S be a p-group and E be a subgroup of S. The operation `FrattiniTest` checks that

$$C_{N_S(E)}(E/\Phi(E)) = E.$$

This test applies at any prime p. It is based on [Gau27, Appendix A].

Example

```
gap> S := SmallGroup(3^4, 7);
<pc group of size 81 with 4 generators>
gap> A := First(MaximalSubgroups(S), A -> IsElementaryAbelian(A));
Group([ f2, f3, f4 ])
gap> B := First(MaximalSubgroups(S), A -> Exponent(A) = 3
>               and not IsElementaryAbelian(A));
Group([ f1, f3, f4 ])
gap> C := First(MaximalSubgroups(S), A -> Exponent(A) = 9);
Group([ f1*f2^2, f3, f4 ])
gap> T := Intersection(A, B);
Group([ f3^2, f4^2 ])
gap> FrattiniTest(S, A);
true
gap> FrattiniTest(S, B);
true
gap> FrattiniTest(S, C);
true
gap> FrattiniTest(S, T);
false
```

3.2.6 LiftTest (for IsPGroup, IsPGroup, IsInt)

▷ `LiftTest(S, E, i)` (operation)

Returns: true or false

Let S be a p-group, E be a subgroup of S of type i (see `RankTest` (3.2.3)). The operation `LiftTest` checks whether the maps in $N_{\mathrm{Out}_{\mathscr{F}}(E)}(\mathrm{Out}_S(E))$ are present as expected. In particular, there is some value a known to divide $N_{\mathrm{Out}_{\mathscr{F}}(E)}(\mathrm{Out}_S(E))$ for each choice of i. We check that a divides:

- $|\mathrm{Aut}(N_S(E))|$;
- $|N_{\mathrm{Aut}(E)}(\mathrm{Aut}_S(E))|$; and
- $|N_{\mathrm{Out}(E)}(\mathrm{Out}_S(E)) : C_{\mathrm{Out}(E)}(\mathrm{Out}_S(E))|$.

This test applies at any prime p. It is based on [Gau27, Lemma A.1] and [OV09, Proposition 3.3(b)].

Example

```
gap> S := SmallGroup(3^6, 479);
<pc group of size 729 with 6 generators>
gap> A := First(MaximalSubgroups(S), IsElementaryAbelian);
Group([ f2, f3, f4, f5, f6 ])
gap> B := First(MaximalSubgroups(S), A -> Exponent(A) = 3
>               and not IsElementaryAbelian(A));
Group([ f1, f3, f4, f5, f6 ])
gap> C := First(MaximalSubgroups(S), A -> Exponent(A) = 9);
Group([ f1, f2, f3, f5, f6 ])
gap> T := Intersection(A, B);
Group([ f3^2, f4^2, f5^2, f6^2 ])
gap> LiftTest(S, A, 0);
true
gap> LiftTest(S, B, 0);
true
gap> LiftTest(S, C, 0);
true
gap> LiftTest(S, T, 1);
false
```

3.2.7 ValidAutTest (for IsPGroup, IsPGroup, IsInt)

▷ `ValidAutTest(S, E, i)` (operation)

Returns: true or false

Let S be a p-group, E be a subgroup of S of type i (see `RankTest` (3.2.3)). The operation `ValidAutTest` checks whether there is a valid section of $\mathrm{Aut}(E)$ that contains the expected $\mathrm{Aut}_{\mathcal{F}}(E)$. Note that this check is up to (order of) composition factors, and does not imply that the expected section actually exists inside $\mathrm{Aut}(E)$.

This test applies at any prime p. It is based on [Sam14, Proposition 6.9] and [PS21, Proposition 4.5].

Example

```
gap> S := SmallGroup(3^4, 7);
<pc group of size 81 with 4 generators>
gap> A := First(MaximalSubgroups(S), A -> IsElementaryAbelian(A));
Group([ f2, f3, f4 ])
gap> B := First(MaximalSubgroups(S), A -> Exponent(A) = 3
>               and not IsElementaryAbelian(A));
Group([ f1, f3, f4 ])
gap> C := First(MaximalSubgroups(S), A -> Exponent(A) = 9);
Group([ f1*f2^2, f3, f4 ])
gap> ValidAutTest(S, A, 0);
true
```

```
gap> ValidAutTest(S, B, 0);
true
gap> ValidAutTest(S, C, 0);
false
```

3.2.8 RadicalTest (for IsPGroup, IsPGroup)

▷ `RadicalTest(`*S*`, `*E*`)` (operation)

Returns: true or false

Let S be a p-group, E be a proto-essential subgroup. The operation `RadicalTest`checks whether

$$|O_p(\mathrm{Aut}(E)) \cap \mathrm{Aut}_S(E)| = [E : Z(E)].$$

This test applies at any prime p. It is based on [PS21, Proposition 4.8(2)].

Example

```
gap> S := SmallGroup(3^4, 7);
<pc group of size 81 with 4 generators>
gap> A := First(MaximalSubgroups(S), A -> IsElementaryAbelian(A));
Group([ f2, f3, f4 ])
gap> B := First(MaximalSubgroups(S), A ->
>               Exponent(A) = 3 and not IsElementaryAbelian(A));
Group([ f1, f3, f4 ])
gap> C := First(MaximalSubgroups(S), A -> Exponent(A) = 9);
Group([ f1*f2^2, f3, f4 ])
gap> RadicalTest(S, A);
true
gap> RadicalTest(S, B);
true
gap> RadicalTest(S, C);
false
```

Chapter 4

Checks for reduced fusion systems

In this chapter, we describe all the tests currently implemented in GAP that test whether a group S *could* support reduced fusion systems. This is done without constructing the fusion systems, and even possible automizers for any fusion system. As such, we can determine whether there exists any reduced fusion system $\mathcal{F}$ on S for a fixed choice of $\mathcal{E}(\mathcal{F})$. We can also determine this for any choice of $\mathcal{E}(\mathcal{F})$.

4.1 The main functions

4.1.1 PossiblySupportsReducedFusionSystems (for IsPGroup)

▷ `PossiblySupportsReducedFusionSystems(S)` (attribute)

Returns: true or false

Given a p-group S, the attribute `PossiblySupportsReducedFusionSystems` checks whether S *could* support reduced fusion systems.

Note that it is not possible to check whether a group can support reduced fusion systems without actually constructing them. This is because the expected automizer sequences that lead to reduced fusion systems may not be saturated, and the only way we can discover this is by checking some radical, non-essential, subgroup in the expected saturated fusion system. Nonetheless, p-groups that pass the tests in this chapter are quite likely to support reduced fusion systems.

This function checks whether the group S satisfies certain properties that are known to hold for a p-group that supports reduced fusion systems. In the process, we compute the proto-essential subgroups of S, along with their automorphism groups. See Section 4.2 for the list of specific tests currently implemented. We do not construct automizer sequences.

Example

```
gap> S := SmallGroup(2^7, 931);
<pc group of size 128 with 7 generators>
gap> PossiblySupportsReducedFusionSystems(S);
true
```

4.1.2 PossiblySupportsReducedFusionSystems_PE (for IsPGroup, IsList)

▷ `PossiblySupportsReducedFusionSystems_PE(S, L)` (operation)

Returns: true or false

Given a p-group S and a list of proto-essential subgroups L, the operation `PossiblySupportsReducedFusionSystems_PE` checks whether there *could* exist a reduced fusion system $\mathcal{F}$ on S where the set of essential subgroups $\mathcal{E}(\mathcal{F})$ is a subset of L. Note that we run `GenerateProtoEssentials` on L (which can, in theory, increase the number of subgroups in L).

Note that it is not possible to check whether a group can support reduced fusion systems without actually constructing them. This is because the expected automizer sequences that lead to reduced fusion systems may not be saturated, and the only way we can discover this is by checking some radical, non-essential, subgroup in the expected saturated fusion system. Nonetheless, p-groups that pass the tests in this chapter are quite likely to support reduced fusion systems.

This function checks whether the group S satisfies certain properties that are known to hold for a p-group that supports reduced fusion systems. We also construct the automorphism group for subgroups in L. See Section 4.2 for the list of specific tests currently implemented. We do not construct automizer sequences.

Example

```
gap> S := SmallGroup(2^7, 931);
<pc group of size 128 with 7 generators>
gap> L := AllProtoEssentials(S);
[ Group([ f7, f1, f2, f4, f5, f6 ]), Group([ f5, f6, f7, f1, f3, f4 ]),
Group([ f5, f6, f7, f2, f3, f4 ]), Group([ f1, f2, f4, f6, f7 ]),
Group([ f1, f3, f5, f6, f7 ]) ]
gap> PossiblySupportsReducedFusionSystems_PE(S, L{[1..3]});
true
gap> PossiblySupportsReducedFusionSystems_PE(S, L{[3..5]});
false
```

4.2 The specific functions

4.2.1 IsDSWGroup (for IsPGroup)

▷ `IsDSWGroup(`S`)` (attribute)

Returns: true or false

Given a p-group S, the attribute `IsDSWGroup` checks whether $p = 2$, and that S is isomorphic to dihedral, semi-dihedral group, or the wreath product $C_{2^a} \wr C_2$.

Using the work of Oliver and his collaborators, there are many properties on a 2-group S that imply that, if S supports reduced fusion systems, then S is isomorphic to one of the three groups (and is called a $\mathcal{DSW}$-group) (see, for example, [AOV13, Proposition 5.2]).

Example

```
gap> S := DihedralGroup(2^3);
<pc group of size 8 with 3 generators>
gap> IsDSWGroup(S);
true
gap> S := SmallGroup(2^4, 8);
<pc group of size 16 with 4 generators>
gap> IsDSWGroup(S);
true
gap> S := QuaternionGroup(2^5);
<pc group of size 32 with 5 generators>
gap> IsDSWGroup(S);
false
```

```
gap> S := WreathProduct(CyclicGroup(IsPermGroup, 2^3), Group((1,2)));
Group([ (1,2,3,4,5,6,7,8), (9,10,11,12,13,14,15,16),
(1,9)(2,10)(3,11)(4,12)(5,13)(6,14)(7,15)(8,16) ])
gap> IdGroup(S);
[128, 67]
gap> IsDSWGroup(S);
true
gap> S := SmallGroup(2^7, 437);
<pc group of size 128 with 7 generators>
gap> StructureDescription(S);
"(C8 x C8) : C2"
gap> IsDSWGroup(S);
false
```

4.2.2 ExponentTest (for IsPGroup)

▷ `ExponentTest(`S`)` (attribute)

Returns: true or false

Let S be a p-group. If S has class c, the attribute `ExponentTest` checks that $Z(S)$ has exponent at most p^t, where

$$t := \left\lceil \frac{c-1}{p-1} \right\rceil .$$

This test applies at any prime p. It is based on [LP10, Theorem A].

Example

```
gap> S := SmallGroup(3^4, 13);
<pc group of size 81 with 4 generators>
gap> StructureDescription(S);
"C3 x (C9 : C3)"
gap> ExponentTest(S);
true
gap> S := SmallGroup(3^4, 14);
<pc group of size 81 with 4 generators>
gap> StructureDescription(S);
"(C9 x C3) : C3"
gap> ExponentTest(S);
false
```

4.2.3 AbelianTest (for IsPGroup)

▷ `AbelianTest(`S`)` (attribute)

Returns: true or false

Let S be a 2-group. The attribute `AbelianTest` first asserts whether there exists an abelian subgroup A of S satisfying $[N_S(A) : A] = 2$ and:

- $N_S(A) = S$; or
- $N_S(A) < S$, with $\mathrm{Aut}_S(A) \not\leq O_2(\mathrm{Aut})(A)$.

If so, the function returns true only if S is a $\mathscr{DSW}$-group.

This test only applies at $p = 2$. It is based on [AOV13, Proposition 5.2(a), (c)].

Example

```
gap> S := DihedralGroup(2^4);
<pc group of size 16 with 4 generators>
gap> AbelianTest(S);
true
gap> S := SmallGroup(2^5, 9);
<pc group of size 32 with 5 generators>
gap> StructureDescription(S);
"(C8 x C2) : C2"
gap> StructureDescription(Center(S));
"C2 x C2"
gap> AbelianTest(S);
false
```

4.2.4 DerivedCyclicTest (for IsPGroup)

▷ `DerivedCyclicTest(`S`)` (attribute)

Returns: true or false

Let S be a p-group. If S' is cyclic, then the attribute `DerivedCyclicTest` returns true only if:

- S is a $\mathscr{DSW}$-group; or
- S has an abelian subgroup of index p (at p odd).

This test applies at any prime p. It is based on [AOV13, Proposition 5.2(b)] (if $p = 2$).

Example

```
gap> S := DihedralGroup(2^4);
<pc group of size 16 with 4 generators>
gap> DerivedCyclicTest(S);
true
gap> S := SmallGroup(2^5, 9);
<pc group of size 32 with 5 generators>
gap> StructureDescription(S);
"(C8 x C2) : C2"
gap> StructureDescription(Center(S));
"C2 x C2"
gap> DerivedCyclicTest(S);
false
```

4.2.5 ValidAutTest (for IsPGroup)

▷ `ValidAutTest(`S`)` (attribute)

Returns: true or false

Let S be a p-group (for p odd). The attribute `ValidAutTest` returns true only if $\mathrm{Aut}(S)$ is not a p-group. If we further know that S has class $\leq p-1$, and $\mathrm{Aut}(S)$ has a Hall p-complement H, we check that $[S,H] = S$.

This test does not apply if $p \leq 3$. It is based on [Cra11, 7.82] and [DGPS11].

Example

```
gap> S := SmallGroup(5^5, 38);
<pc group of size 3125 with 5 generators>
```

```
gap> NilpotencyClassOfGroup(S);
4
gap> AutomorphismGroup(S);
<group of size 15625 with 6 generators>
gap> ValidAutTest(S);
false
gap> S := SmallGroup(5^5, 30);
<pc group of size 3125 with 5 generators>
gap> NilpotencyClassOfGroup(S);
4
gap> AutomorphismGroup(S);
<group of size 1250000 with 11 generators>
gap> ValidAutTest(S);
true
gap> S := SmallGroup(5^5, 32);
<pc group of size 3125 with 5 generators>
gap> NilpotencyClassOfGroup(S);
4
gap> AutomorphismGroup(S);
<group of size 62500 with 8 generators>
gap> ValidAutTest(S);
false
```

4.2.6 OmegaAbelianizationTest (for IsPGroup)

▷ `OmegaAbelianizationTest(`S`)` (attribute)

Returns: true or false

Let S be a 2-group, $D := S'$ and $O := \Omega(Z(S))$. The attribute `OmegaAbelianizationTest` returns false only if:

- $[\Omega(Z(S)) : \Omega(Z(S)) \cap D] = 2$; or
- $\Omega(Z(S)) \not\leq D$ if $\mathrm{Aut}(S)$ is a 2-group.

This test only applies at $p = 2$. It is based on [AKO11, Corollary I.8.5].

Example

```
gap> S := DihedralGroup(2^4);
<pc group of size 16 with 4 generators>
gap> OmegaAbelianizationTest(S);
true
gap> S := SmallGroup(2^4, 3);
<pc group of size 16 with 4 generators>
gap> StructureDescription(S);
"(C4 x C2) : C2"
gap> OmegaAbelianizationTest(S);
false
```

4.2.7 Transfer test for a group

▷ `TransferMaximalTest(`S`)` (attribute)

▷ `TransferFpMaximalTest(`S`)` (attribute)

Returns: true or false

Let S be a 2-group. The attribute `TransferMaximalTest` checks that, if $\mathrm{Aut}(S)$ is a 2-group, then

$$\bigcap_{M\in\mathscr{M}(S)} \ker(\mathrm{tr}_M^S) = S',$$

where $\mathscr{M}(S)$ is the set of maximal subgroups of S, and $\mathrm{tr}_M^S: S \to M/M'$ is the transfer homomorphism. Alternatively, `TransferFpMaximalTest` checks that, if $\mathrm{Aut}(S)$ is a 2-group, then

$$\bigcap_{M\in\mathscr{M}(S)} \ker(\mathrm{tr}'^S_M) = \Phi(S),$$

where $\mathscr{M}(S)$ is the set of maximal subgroups of S, and $\mathrm{tr}'^S_M: S \to M/\Phi(M)$ is the transfer homomorphism.

These tests only apply at $p = 2$ when $\mathrm{Aut}(S)$ is a 2-group. It is based on [AOV17, Lemma 2.1].

Example

```
gap> S := SmallGroup(2^7, 903);
<pc group of size 128 with 7 generators>
gap> TransferMaximalTest(S);
false
gap> TransferFpMaximalTest(S);
true
```

4.2.8 Transfer test with a list

▷ `TransferTest(S, L)` (operation)

▷ `TransferFpTest(S, L)` (operation)

Returns: true or false

Let S be a 2-group and L be a list of (proto-essential) subgroups of S. The operation `TransferTest` checks that, if $\mathrm{Aut}(S)$ is a 2-group, then

$$\bigcap_{M\in L} \ker(\mathrm{tr}_M^S) = S',$$

where $\mathrm{tr}_M^S: S \to M/M'$ is the transfer homomorphism. Alternatively, `TransferFpTest` checks that, if $\mathrm{Aut}(S)$ is a 2-group, then

$$\bigcap_{M\in L} \ker(\mathrm{tr}'^S_M) = \Phi(S),$$

where $\mathrm{tr}'^S_M: S \to M/\Phi(M)$ is the transfer homomorphism.

These tests only apply at $p = 2$ when $\mathrm{Aut}(S)$ is a 2-group. It is based on [AOV17, Lemma 2.1].

Example

```
gap> S := SmallGroup(2^7, 931);
<pc group of size 128 with 7 generators>
gap> L := AllProtoEssentials(S);
[ Group([ f7, f1, f2, f4, f5, f6 ]), Group([ f5, f6, f7, f1, f3, f4 ]),
Group([ f5, f6, f7, f2, f3, f4 ]), Group([ f1, f2, f4, f6, f7 ]),
Group([ f1, f3, f5, f6, f7 ]) ]
gap> TransferTest(S, L{[1..3]});
true
gap> TransferTest(S, L{[3..5]});
false
```

4.2.9 C0Test (for IsPGroup, IsList)

▷ `C0Test(S, L)` (operation)

Returns: true or false

Let S be a p-group, and $L := \mathfrak{C}_0$. The operation `C0Test` checks that there exists some subgroup E in L such that $\Omega(Z(S))$ is not semi-characteristic in E, i.e. invariant under $O^p(O^{p'}(\mathrm{Aut}(E)))$.

This test applies at any prime p. It is based on [Gau26, Corollary 3.12].

Example

```
gap> L := AllSmallGroups(3^5, A -> NilpotencyClassOfGroup(A) >= 3);;
gap> Length(L);
32
gap> Number(L, S -> C0Test(S, MainProtoEssentials(S, 0)));
17
```

4.2.10 CTest (for IsPGroup, IsList)

▷ `CTest(S, L)` (operation)

Returns: true or false

Let S be a p-group, and let L be a list of proto-essential subgroups of S. The operation `CTest` checks that:

- L has at least 2 elements, or
- L consists of a single subgroup, which is not normal in S, and $\mathrm{Aut}(S)$ is not a p-group.

This test applies at any prime p. It is based on [AOV17, Lemma 2.2(e)].

Example

```
gap> L := AllSmallGroups(3^5, A -> NilpotencyClassOfGroup(A) >= 3);;
gap> Length(L);
32
gap> Number(L, S -> CTest(S, AllProtoEssentials(S)));
16
```

4.2.11 SemiCharacteristicGeneratedTest (for IsPGroup, IsList)

▷ `SemiCharacteristicGeneratedTest(S, L)` (operation)

Returns: true or false

Let S be a p-group, and let L be a list of proto-essential subgroups of S. The operation `SemiCharacteristicGeneratedTest` checks that the subgroup

$$\langle [O^p(\mathrm{Aut}(S)), S], Q_E \mid E \in L \rangle = S,$$

where Q_E is the semi-characteristic closure of $[N_S(E), E]$ in E.

This test applies at any prime p. It is based on [AOV17, Lemma 2.2(f)].

Example

```
gap> S := SmallGroup(2^7, 931);
<pc group of size 128 with 7 generators>
gap> L := AllProtoEssentials(S);
[ Group([ f7, f1, f2, f4, f5, f6 ]), Group([ f5, f6, f7, f1, f3, f4 ]),
Group([ f5, f6, f7, f2, f3, f4 ]), Group([ f1, f2, f4, f6, f7 ]),
```

```
Group([ f1, f3, f5, f6, f7 ]) ]
gap> SemiCharacteristicGeneratedTest(S, L{[1..3]});
true
gap> SemiCharacteristicGeneratedTest(S, L{[3..5]});
false
```

4.2.12 CoreTest (for IsPGroup, IsList)

▷ `CoreTest(`*S*`, `*L*`)` (operation)

Returns: true or false

Let S be a p-group, and let L be a list of proto-essential subgroups of S. The operation `CoreTest` checks that if N is a normal subgroup of S such that N is semi-characteristic (i.e. fixed by $O^p(O^{p'}(\mathrm{Aut}(E)))$) in each $E \in L$, then $N = 1$.

This test applies at any prime p.

Example

```
gap> S := SmallGroup(256, 6366);
<pc group of size 256 with 8 generators>
gap> L := AllProtoEssentials(S);
[ Group([ f1, f2, f4, f5, f6, f7, f8 ]), Group([ f7, f8, f3, f4, f5, f6 ]) ]
gap> C := List(L, CharacteristicSubgroups);;
gap> Intersection(C);
[ Group([  ]) ]
gap> CoreTest(S, L);
false
```

Chapter 5

Automizer Sequences

Let S be a p-group, with proto-essential subgroups $E_1, \ldots, E_n$. An *automizer sequence* is a list $P := [H_{i_1}, \ldots, H_{i_k}]$, where $H_{i_j} \leq \mathrm{Aut}(E_{i_j})$ is a valid automizer for E_{i_j} in S. In other words, $\mathrm{Aut}_S(E_{i_j})$ is a Sylow p-subgroup of H_{i_j}, and $H_{i_j}/\mathrm{Inn}(E_{i_j})$ contains a strongly p-embedded subgroup. Note that we do not include a choice of $H_S \leq \mathrm{Aut}(S)$ here. These are prototypes for saturated fusion systems. In this chapter, we describe processes to determine many important properties about automizer sequences that will allow us to determine whether the corresponding fusion system would be reduced.

5.1 The functions

5.1.1 FindReducedSequences (for IsPGroup, IsList)

▷ `FindReducedSequences(S, L)` (operation)

Returns: a list of lists of integers

Let S be a p-group, and let L be a list of subgroups of S, all of which are proto-essential in S. We compute all possible automizer sequences involving the subgroups in L that could lead to reduced fusion systems. Each sequence is returned as a list A of integers, with one entry for each subgroup E_i in L: $A[i] = 0$ if E_i is not essential, and otherwise $A[i]$ is the position of $\mathrm{Aut}_{\mathscr{F}}(E_i)$ in `ValidAutomizers(S, E_i)`.

5.1.2 IsomorphismSequences (for IsPGroup, IsList, IsList)

▷ `IsomorphismSequences(S, P1, P2)` (operation)

Returns: an automorphism of S, or `fail`

Given two automizer sequences P_1 and P_2, check whether there exists some $\phi \in \mathrm{Aut}(S)$ such that $P_1^\phi = P_2$. This computation mimics isomorphism of fusion systems. The action of ϕ depends on the ordering of the P_i's.

5.1.3 ValidAutomizers (for IsPGroup, IsPGroup)

▷ `ValidAutomizers(S, E)` (operation)

Returns: a list

Finds valid automizers for E in S. This is not part of the proto-essentials test and forms the first step in the actual construction of fusion systems. Nonetheless, if a subgroup is proto-essential, then it

must have a valid automizer. This function does not construct all subgroups of $\mathrm{Aut}(E)/O_p(\mathrm{Aut}(E))$, and instead uses iterative maximal subgroup condition to do so.

5.1.4 FindSmallestBorel (for IsPGroup, IsGroupOfAutomorphismsFiniteGroup)

▷ `FindSmallestBorel(`*S, AutFE*`)` (operation)

Returns: a group

Let S be a p-group, and let $E < S$ with some fixed $H_E \leq \mathrm{Aut}(E)$ part of an automizer sequence. The operation `FindSmallestBorel` computes the subgroup of $\mathrm{Aut}(S)$ consisting of extensions from $N_{H_E}(\mathrm{Aut}_S(E))$. If not every map in the normalizer subgroup exists, we return `fail`.

5.1.5 IsAutomizerBorelCompatible (for IsPGroup, IsGroupOfAutomorphismsFiniteGroup, IsGroupOfAutomorphismsFiniteGroup)

▷ `IsAutomizerBorelCompatible(`*S, AutFS, AutFE*`)` (operation)

Returns: true or false

Let S be a p-group, and let $E < S$ with some fixed $H_E \leq \mathrm{Aut}(E)$ part of an automizer sequence. Let $H_S \leq \mathrm{Aut}(S)$ be a subgroup. The operation `IsAutomizerBorelCompatible` checks whether the restrictions of H_S are *compatible* with H_E. In particular, if $A_0 \leq \mathrm{Aut}(E)$ consists of restrictions from H_S, we have that

$$O^{p'}(\langle A_0, H_E \rangle) = H_E.$$

5.1.6 IsCompatibleSequence (for IsPGroup, IsList)

▷ `IsCompatibleSequence(`*S, P*`)` (operation)

Returns: true or false

Let S be a p-group and let P be an automizer sequence. The operation `IsCompatibleSequence` checks whether P can lead to a saturated fusion system. Note that there can be false positives -- P may not lead to a saturated fusion system (based on the analysis of centric, radical subgroups) but passes this test.

Chapter 6

Other functions

6.1 Group actions

6.1.1 OnQuotient

▷ `OnQuotient(q)` (function)

Returns: a function

Let G be a finite group, and A be a normal subgroup of G. Let $q: G \to G/A$. The function `OnQuotient(q)` returns a function, such that for $t \in G/A$ and $x \in G$, `OnQuotient(q)(t, x)` returns $q(x^{-1}hx)$, where $h \in G$ satisfies $q(h) = t$.

6.1.2 CentralizerMod (for IsGroup, IsGroup, IsGroup)

▷ `CentralizerMod(G, A, B)` (operation)

Returns: a subgroup of G

Let G be a finite group, with subgroups $B \leq A \leq G$, where B is normal in G. The operation `CentralizerMod(G, A, B)` returns the subgroup $C_G(A/B)$, given by:

$$C_G(A/B) := \langle x \in G \mid [A,x] \leq B \rangle.$$

Example

```
gap> S := SmallGroup(5^6, 643);
<pc group of size 15625 with 6 generators>
gap> U := UpperCentralSeries(S);
[ Group([ a6, a5, a4, a3, a1, a2 ]), Group([ a6, a5, a4, a3 ]),
Group([ a6, a5, a4 ]), Group([ a6, a5 ]), Group([ a6 ]), Group([  ]) ]
gap> R := CentralizerMod(S, U[4], U[6]);
Group([ <identity> of ..., a2, a3, a4, a5, a6 ])
gap> R = Centralizer(S, U[4]);
true
gap> A := CentralizerMod(S, U[3], U[5]);
Group([ a1, <identity> of ..., a3, a4, a5, a6 ])
gap> A = R;
false
```

6.1.3 OnImage

▷ `OnImage(x, phi)` (function)

Returns: an element or a subset

Let ϕ be a map and let x be either an element or a subset of the domain of ϕ. The function `OnImage(x, phi)` returns $\phi(x)$.

6.1.4 OnImageNM

▷ `OnImageNM(n)` (function)

Returns: a function

Let G be a group and n a nice monomorphism for (a subgroup of) $\mathrm{Aut}(G)$, with image A. The function `OnImageNM(n)` returns a function such that, for some element $\phi \in A$ and an element (or a subgroup) x of G, `OnImageNM(n)(x, phi)` return $\hat{\phi}(x)$, where $\hat{\phi} \in \mathrm{Aut}(G)$ is the preimage of ϕ with respect to the injective map n.

Example

```
gap> S := SmallGroup(5^6, 642);
<pc group of size 15625 with 6 generators>
gap> AutS := AutomorphismGroup(S);
<group of size 1562500 with 10 generators>
gap> n := NiceMonomorphism(AutS);
<action isomorphism>
gap> AutS := n(AutS);
<permutation group of size 1562500 with 10 generators>
gap> V := Group(S.2, S.6);
<pc group with 2 generators>
gap> A := Stabilizer(AutS, V, OnImageNM(n));
<permutation group of size 12500 with 7 generators>
```

6.1.5 OnImageTuples

▷ `OnImageTuples(L, phi)` (function)

Returns: a list

Let ϕ be a map and let L be a list of elements in the domain of ϕ. The function `OnImageTuples(L, phi)` returns a list containing $\phi(x)$ for each $x \in L$, while maintaining the order.

6.1.6 OnImageTuplesNM

▷ `OnImageTuplesNM(n)` (function)

Returns: a function

Let G be a group and n a nice monomorphism for (a subgroup of) $\mathrm{Aut}(G)$, with image A. The function `OnImageTuplesNM(n)` returns a function such that, for some element $\phi \in A$ and a list L of elements in G, `OnImageTuplesNM(n)(L, phi)` return a list of elements $\hat{\phi}(x)$, where $x \in L$ and $\hat{\phi} \in \mathrm{Aut}(G)$ is the preimage of ϕ with respect to the injective map n.

6.1.7 OnPreImage

▷ `OnPreImage(q)` (function)

Returns: a function

Let A be a group that acts on the set Ω with kernel K, and let $q:A \to G$ be a group homomorphism with $\ker(q) \leq K$. The function `OnPreImage(q)` returns a function, such that, for some element $g \in G$ and $x \in \Omega$, `OnPreImage(q)(x, g)` returns x^a, where $a \in A$ is the preimage of g with respect to q.

6.1.8 OnPreImageTuples

▷ `OnPreImageTuples(q)` (function)

Returns: a function

Let A be a group that acts on the set Ω with kernel K, and let $q:A \to G$ be a group homomorphism with $\ker(q) \leq K$. The function `OnPreImageTuples(q)` returns a function, such that, for some element $g \in G$ and a subset $L \subseteq \Omega$, `OnPreImageTuples(q)(L, g)` returns a list consisting of x^a for $x \in L$, where $a \in A$ is the preimage of g with respect to q.

6.2 Group automorphisms

6.2.1 Automizer (for IsGroup, IsGroup)

▷ `Automizer(G, H)` (operation)

▷ `Automizer(A, H)` (operation)

Returns: a subgroup of Aut(H)

Let G and H be subgroups of some finite group. The group `Automizer(G, H)` is the automizer subgroup $\mathrm{Aut}_G(H)$, given by:

$$\mathrm{Aut}_G(H) := \langle c_g \mid g \in N_G(H) \rangle,$$

where $c_g:H \to H$ is the conjugation automorphism. It is a subgroup of $\mathrm{Aut}(H)$.

Example

```
gap> G := AlternatingGroup(6);
Alt( [ 1 .. 6 ] )
gap> S := SylowSubgroup(G, 2);
Group([ (1,2)(3,4), (1,3)(2,4), (1,2)(5,6) ])
gap> M := MaximalSubgroups(S);
[ Group([ (3,4)(5,6), (1,2)(3,4) ]), Group([ (1,3)(2,4), (1,2)(3,4) ]),
Group([ (1,3,2,4)(5,6), (1,2)(3,4) ]) ]
gap> AutGM := List(M, A -> Automizer(G, A));
[ <group with 2 generators>, <group with 2 generators>,
<group with 2 generators> ]
gap> List(AutGM, StructureDescription);
[ "S3", "S3", "C2" ]
```

Alternatively, if A is a subgroup of $\mathrm{Aut}(G)$, then `Automizer(A, H)` returns the subgroup $\mathrm{Aut}_A(H)$ of $\mathrm{Aut}(H)$ given by:

$$\mathrm{Aut}_A(H) := \langle \alpha|_H \mid \alpha \in A, \alpha(H) = H \rangle.$$

Example

```
gap> S := SmallGroup(5^6, 642);
<pc group of size 15625 with 6 generators>
gap> V := Group(S.2, S.6);
<pc group with 2 generators>
```

```
gap> AutS := AutomorphismGroup(S);
<group of size 1562500 with 10 generators>
gap> AutSV := Automizer(AutS, V);
<group with 7 generators>
gap> AutV := AutomorphismGroup(V);;
gap> n := NiceMonomorphism(AutV);
<action isomorphism>
gap> AutV := n(AutV);;
gap> Size(AutV);
480
gap> AutSV := n(AutSV);
<permutation group with 7 generators>
gap> Index(AutV, AutSV);
24
gap> Size(AutSV);
20
```

6.2.2 Restriction of homomorphisms

▷ `RestrictedHomomorphismOnto(phi, X)` (operation)
▷ `RestrictedAutomorphism(phi, X)` (operation)

Returns: a group automorphism

Let $\phi: A \to B$ be a group homomorphism, and let $X \leq A$. The operation `RestrictedHomomorphismOnto(phi, X)` returns the surjective group homomorphism $\phi|_X: X \to \phi(X)$ given by restriction. If ϕ is also an isomorphism that fixes X, then the operation `RestrictedAutomorphism(phi, X)` returns the group automorphism $\phi|_X: X \to \phi(X)$ given by restriction.

6.2.3 Transfer homomorphism

▷ `TransferHomomorphism(G, S)` (operation)
▷ `TransferHomomorphismFp(G, S)` (operation)

Returns: a homomorphism

Let G be a finite group and $S \leq G$. The operation `TransferHomomorphism(G, S)` returns the transfer homomorphism $\mathrm{tr}_S^G: G \to S/S'$. Moreover, if S is a p-group, then the operation `TransferHomomorphismFp(G, S)` returns the transfer homomorphism $\mathrm{tr}'^G_S: G \to S/\Phi(S)$.

6.3 Semi-characteristic subgroups

Let S be a p-group and $A \leq S$. We say that A is *semi-characteristic* in S if

- A is normal in S, and
- A is fixed by $O^p(O^{p'}(\mathrm{Aut}(S)))$.

6.3.1 SemiAutomorphismGroup (for IsPGroup)

▷ `SemiAutomorphismGroup(S)` (attribute)

Returns: a subgroup of Aut(S)

Let S be a p-group. The operation SemiAutomorphismGroup returns the semi-automorphism group of S, which is given by $\langle \mathrm{Inn}(S), O^p(O^{p'}(\mathrm{Aut}(S)))\rangle$.

6.3.2 SemiCharacteristicSubgroups (for IsPGroup)

▷ SemiCharacteristicSubgroups(*S*) (attribute)

Returns: a list

Let S be a p-group. The attribute SemiCharacteristicSubgroups returns all the semi-characteristic subgroups of S. These are the normal subgroups of S fixed by $O^p(O^{p'}(\mathrm{Aut}(S)))$.

6.3.3 IsSemiCharacteristic (for IsPGroup, IsPGroup)

▷ IsSemiCharacteristic(*S*, *A*) (operation)

Returns: true or false

Let S be a p-group and A be a subgroup of S. The operation IsSemiCharacteristic checks whether A is semi-characteristic in S. That is, we check whether:

- A is normal in S, and
- A is fixed by $O^p(O^{p'}(\mathrm{Aut}(S)))$.

6.3.4 SemiCharacteristicClosure (for IsPGroup, IsPGroup)

▷ SemiCharacteristicClosure(*S*, *A*) (operation)

Returns: a subgroup of S

Let S be a p-group and A be a subgroup of S. The operation SemiCharacteristicClosure returns the smallest semi-characteristic subgroup of S containing A. This is the intersection of all semi-characteristic subgroups of S containing A.

Example

```
gap> S := SmallGroup(2^8, 6331);
<pc group of size 256 with 8 generators>
gap> E1 := Group(S.1, S.2, S.4, S.5, S.6, S.7, S.8);
<pc group with 7 generators>
gap> E2 := Group(S.1, S.2, S.4, S.5, S.7, S.8);
<pc group with 6 generators>
gap> A := Group(S.5*S.8);
<pc group with 1 generator>
gap> AutomorphismGroup(E1);
<group of size 6144 with 12 generators>
gap> AutomorphismGroup(E2);
<group of size 12288 with 13 generators>
gap> IsCharacteristicSubgroup(E1, A);
false
gap> IsCharacteristicSubgroup(E2, A);
false
gap> IsPGroup(E1);
true
gap> IsPGroup(E2);
true
gap> SemiAutomorphismGroup(E1);
```

```
<group of size 384 with 8 generators>
gap> SemiAutomorphismGroup(E2);
<group of size 3072 with 11 generators>
gap> IsPGroup(A);
true
gap> IsSemiCharacteristic(E1, A);
true
gap> IsSemiCharacteristic(E2, A);
false
gap> SemiCharacteristicClosure(E2, A);
Group([ f4, f5, f8, f4*f5, f4*f8, f5*f8, f4*f5*f8 ])
```

References

[AKO11] M. Aschbacher, R. Kessar, and B. Oliver. *Fusion systems in algebra and topology*, volume 391. Cambridge University Press, 2011. 17

[AOV13] K. Andersen, B. Oliver, and J. Ventura. Fusion systems and amalgams. *Mathematische Zeitschrift*, 274(3):1119–1154, 2013. 14, 15, 16

[AOV17] K. Andersen, B. Oliver, and J. Ventura. Reduced fusion systems over 2-groups of small order. *Journal of Algebra*, 489:310–372, 2017. 8, 18, 19

[Cra11] D. Craven. *The theory of fusion systems: An algebraic approach*, volume 131. Cambridge University Press, 2011. 16

[DGPS11] Antonio Díaz, Adam Glesser, Sejong Park, and Radu Stancu. Tate's and yoshida's theorems on control of transfer for fusion systems. *Journal of the London Mathematical Society*, 84(2):475–494, 2011. 16

[Gau26] Pete Gautam. Fusion systems on Sylow 3-subgroups of Fischer and Monster sporadic groups – II. *arXiv preprint arXiv*, 2026. 5, 19

[Gau27] Pete Gautam. Fusion systems on Sylow 3-subgroups of Fischer and Monster sporadic groups – I. *Journal of Algebra*, 710:810–859, 2027. 10, 11

[LP10] J. Lynd and S. Park. Analogues of Goldschmidt's thesis for fusion systems. *Journal of Algebra*, 324(12):3487–3493, 2010. 15

[OV09] B. Oliver and J. Ventura. Saturated fusion systems over 2-groups. *Transactions of the American Mathematical Society*, 361(12):6661–6728, 2009. 10, 11

[PS21] C. Parker and J. Semeraro. Algorithms for fusion systems with applications to p-groups of small order. *Mathematics of Computation*, 90(331):2415–2461, 2021. 11, 12

[Sam14] B. Sambale. *Blocks of finite groups and their invariants*, volume 2127. Springer, 2014. 9, 11

Index

AbelianTest
  for IsPGroup, 15
AllProtoEssentials
  for IsPGroup, 5
Automizer
  for IsGroup, IsGroup, 25
  for IsGroupOfAutomorphismsFiniteGroup, IsGroup and IsFinite, 25

COTest
  for IsPGroup, IsList, 19
CentralizerMod
  for IsGroup, IsGroup, IsGroup, 23
CoreTest
  for IsPGroup, IsList, 20
CTest
  for IsPGroup, IsList, 19

DerivedCyclicTest
  for IsPGroup, 16

ExponentTest
  for IsPGroup, 15

FindReducedSequences
  for IsPGroup, IsList, 21
FindSmallestBorel
  for IsPGroup, IsGroupOfAutomorphismsFiniteGroup, 22
FrattiniTest
  for IsPGroup, IsPGroup, 10

GenerateProtoEssentials
  for IsPGroup, IsList, 6

IsAutomizerBorelCompatible
  for IsPGroup, IsGroupOfAutomorphismsFiniteGroup, IsGroupOfAutomorphismsFiniteGroup, 22
IsCompatibleSequence
  for IsPGroup, IsList, 22
IsDSWGroup
  for IsPGroup, 14
IsomorphismSequences
  for IsPGroup, IsList, IsList, 21
IsProtoEssential
  for IsPGroup, IsPGroup, 7
IsProtoEssentialNC
  for IsPGroup, IsPGroup, 8
IsSemiCharacteristic
  for IsPGroup, IsPGroup, 27

LiftTest
  for IsPGroup, IsPGroup, IsInt, 10

MainProtoEssentials, 6
ModuleTest
  for IsPGroup, IsPGroup, 9

OmegaAbelianizationTest
  for IsPGroup, 17
OnImage, 24
OnImageNM, 24
OnImageTuples, 24
OnImageTuplesNM, 24
OnPreImage, 24
OnPreImageTuples, 25
OnQuotient, 23

PossiblySupportsReducedFusionSystems
  for IsPGroup, 13
PossiblySupportsReducedFusionSystems_PE
  for IsPGroup, IsList, 13

RadicalTest
  for IsPGroup, IsPGroup, 12
RankTest
  for IsPGroup, IsPGroup, 9
RestrictedAutomorphism

for IsGroupHomomorphism and IsBijective,
IsGroup, 26
`RestrictedHomomorphismOnto`
for IsGroupHomomorphism, IsGroup, 26

`SemiAutomorphismGroup`
for IsPGroup, 26
`SemiCharacteristicClosure`
for IsPGroup, IsPGroup, 27
`SemiCharacteristicGeneratedTest`
for IsPGroup, IsList, 19
`SemiCharacteristicSubgroups`
for IsPGroup, 27

`TransferFpMaximalTest`
for IsPGroup, 17
`TransferFpTest`
for IsPGroup, IsList, 18
`TransferHomomorphism`
for IsGroup, IsGroup, 26
`TransferHomomorphismFp`
for IsGroup, IsPGroup, 26
`TransferMaximalTest`
for IsPGroup, 17
`TransferTest`
for IsPGroup, IsList, 18

`ValidAutomizers`
for IsPGroup, IsPGroup, 21
`ValidAutTest`
for IsPGroup, 16
for IsPGroup, IsPGroup, IsInt, 11

`Zed0`
for IsPGroup, 8
`Zed0Test`
for IsPGroup, IsPGroup, 8